Rapport Projet de Programmation :

Faire une brève description du jeu ainsi que des fonctionnalités de base :

Le Cube de Mayhem est un jeu de stratégie au tour par tour prenant place sur une grille carrée représentant un plateau de jeu. Le joueur doit y placer, déplacer ou supprimer des tuiles pour compléter des objectifs. Chaque tuile possède un type et un effet spécifique qui peut influencer la suite de la partie.

Le jeu tire son nom de son aspect « chaotique » : les tuiles interagissent avec le cube, certains événements surviennent de manière imprévisible, et le joueur doit sans cesse s'adapter. Cela en fait un jeu à la fois tactique et dynamique.

Fonctionnalités de base :

Grille de jeu (plateau) :
Le jeu se déroule sur une grille de taille fixe (par exemple, 6x6 ou 10x10).
Chaque case peut contenir une tuile ou rester vide.

Tuiles à placer :
Le joueur dispose d'un ensemble de tuiles sélectionnables.
Les tuiles sont de différents types (ex. : direction, portail, slide, téléporteur, mur, etc.), chacun avec ses propriétés ou interactions.

Placement et suppression :
Le joueur peut placer une tuile depuis la barre de sélection vers une case vide de la grille.
Il peut aussi supprimer une tuile déjà posée pour en changer la configuration ou libérer de la place.

Interaction entre tuiles :
Certaines tuiles déclenchent des effets qui modifient là à direction du cube . Des chaînes de réaction peuvent se produire selon les placements effectués.

Tour de jeu ou logique de tour :
Le jeu peut se jouer avec un système de niveaux. Aussi tous le niveau finis, on passe au suivant l'aide du panel de nouveaux, les niveaux sont accessibles tout le temps pour notre projet.

Objectifs ou score :
Le jeu peut inclure des objectifs spécifiques (atteindre une configuration donnée, survivre un certain nombre de tours, marquer un score).
Un système de points peut récompenser les actions efficaces ou stratégiques.

Interface utilisateur (UI) :
Barre de sélection en bas avec les tuiles disponibles.
Affichage du plateau au centre ou en haut de l'écran.

2 visuels différents :
un visuel entièrement en 2D
ou un visuel 3D après le parseur en 2D

Méthodologie utilisée:

Dès le début nous nous sommes concentré sur la création d'un visuel 3D (3 personnes) et en parallèle les débuts du model (2 personnes)
Chaque fois nous avons séparé la logique du visuel comme le veut le MVC,
la conception de la 3D mettant beaucoup de temps, nous avons commencé un visuel 2D pour avancer sur la logique sans être limité. (3 personnes sur le 2D) (2 personnes sur le 3D).
Enfin la 3D fonctionnelle, on a pu lier la logique à la 3D en conservant un visuel 2D pour toujours avancer en parallèle. Résultant dans un visuel 3D fonctionnel bien que simple.
Au fur et à mesure on a pu implémenter de nouvelles fonctionnalités de la 2D vers la 3D en utilisant le même modèle, seulement un GameView3D et un GameView (2D).

Enoncer les fonctionnalités implémentées :

Éditeur de niveaux : permet de créer un fichier Json à partir d'un niveau créé par l'utilisateur, ce level est ensuite jouable avec le plateau et les tuiles définies.

Moteur de rendu 2D : choix du niveau, parseur puis après validation du joueur, lancement du jeu

Moteur de rendu 3D : après le parseur en 2D, affiche le jeu en 3D sans bibliothèques à partir de affineTransform, comprenant les animations du cube.

Enoncer les fonctionnalités que l'on voulait implémenter mais que l'on n'a pas fait par manque de temps ou de connaissance :

– Finir un niveau à partir d'un certain temps, sinon gameOver.

- Tuiles spéciales avec effet aléatoire.

– Mode qui aide l'utilisateur au fur et à mesure qu'il place les tuiles

L'architecture du projet suit le modèle MVC (Modèle-Vue-Contrôleur) afin de séparer clairement les responsabilités.

- Modèle : Gère l'état du jeu et la logique métier. Les classes principales incluent GameState, Board, Cube, et GameEngine.
- Vue : Responsable de l'affichage et de l'interaction utilisateur. Les composants clés sont GamePanel, MapPanel, et d'autres panneaux d'interface.
- Contrôleur : Coordonne le flux du jeu et les entrées utilisateur. Les classes GameController, InputController, et GameLoop assurent cette gestion.

Le modèle encapsule l'état du jeu via la classe GameState, qui contient la configuration du plateau (Board), la position et l'état du cube (Cube), ainsi que la direction actuelle. La classe GameEngine gère la logique du jeu, notamment les déplacements, les interactions entre le cube et le plateau, et les conditions de victoire ou de défaite.

La vue est composée de plusieurs panneaux Swing, dont GamePanel qui constitue l'interface principale. Elle affiche le plateau, les tuiles disponibles, et gère les interactions utilisateur telles que la sélection des tuiles et le démarrage ou l'arrêt du jeu. Les panneaux affichent également les messages de victoire ou de défaite.

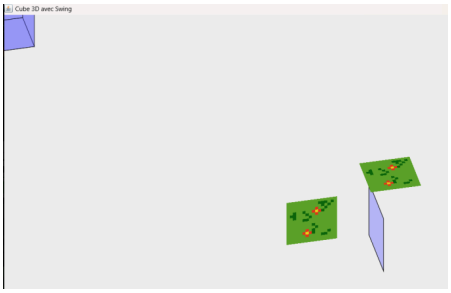
Le contrôleur principal, GameController, orchestre la communication entre le modèle et la vue. Il gère la boucle de jeu via GameLoop et délègue la gestion des entrées clavier et souris à InputController. Cette organisation permet une gestion fluide du déroulement du jeu et des interactions utilisateur.

Le projet met en œuvre une architecture MVC claire, facilitant la maintenance et l'évolution. L'utilisation de classes dédiées pour la logique du jeu (GameEngine) et l'état (GameState) permet une séparation nette des responsabilités. L'interface utilisateur est conçue pour être intuitive et réactive grâce à Swing.

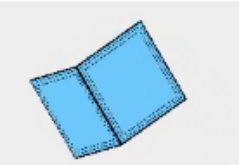
Les différents défis rencontrés :

– La gestion des interactions complexes entre le cube et le plateau a nécessité plusieurs ajustements, notamment l'interaction du board avec le cube et les différentes méthodes du Model.

-La mise en place d'une interface utilisateur fluide a également demandé une attention particulière.



-La création 3D du cube avec AffineTransform a posé de nombreux problèmes car il eut été important de notifier que les plus grands trépas se trouvaient dans la compréhension de la tâche qui nous incombait. Car la composition matricielle était pour nous une nouveauté des plus complexes.



-Des difficultés très chronophages pour le calcul des faces visibles sur le cube, ou les faces visibles n’étaient jamais les bonnes avec de

nombreuses raisons possibles, on a finalement réussi à corriger ces problèmes majeurs, il reste des légers problèmes qui ne gênes pas la compréhension probablement dû à l'imprécision des types “double”. Néanmoins enrichissant pour notre culture vidéoludique. Notamment dans la projection de forme 3d en plan 2d.

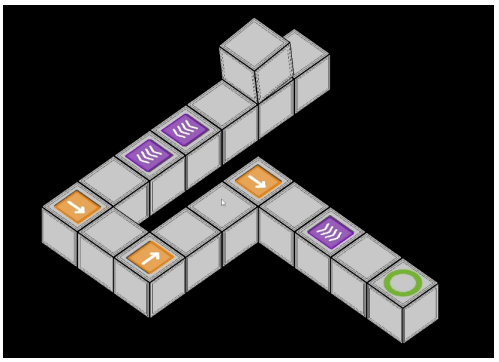
Documentation extérieure :

- Dès le début on a passé de nombreuses heures en vain à faire des recherches pour générer le visuel d’un cube en 3D grâce aux méthodes shear() de AffineTransform, on a pu avoir un visuel mais difficile de faire la moindre animation après.

- Sous conseil de monsieur Padovani on s’est renseigné sur le constructeur AffineTransform, et après recherches on a pu faire un cube avec des animations de translation et rotation à partir de calcul sur des matrices 4x4

- Il nous a ensuite donné la documentation mathématique pour ce qui concerne les calculs avancés de visibilités des faces du cube (pour ne pas voir les faces arrières)

Fonctionnement de la 3D :

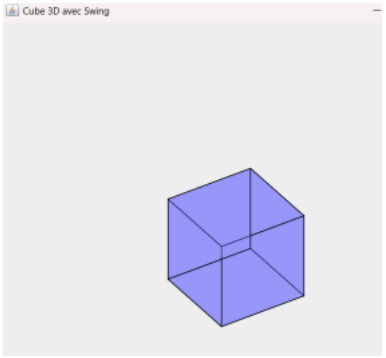


- Pour le fonctionnement de la 3d nous nous sommes inspirés des cours de master 1 de Monsieur Padovani. La class Coord3d est la base elle permet de placer des points dans un plan en 3 dimensions. Elle comporte des fonctions de mathématique vectoriel de base comme le produit scalaire, le produit vectoriel etc. A partir de cette classe on créait la class polygon permettant de créer des polygon 3d des faces de notre cube par exemple, cette class comporte également un arg normal permettant les calculs des faces cachées.
A partir des ses classes la construction de la classe polyhedron (cube).nous permet de créer le cube en 3d.

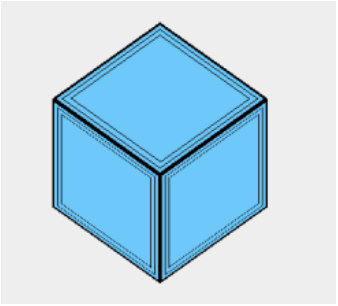
À côté de cela la class Transform3d permet des applications de matrice 4x4 à nos polygon/ polyhedron, contenant notamment les fonctions d’application de matrice au polygon, le calcul de matrice (create rotation, create translation), la composition de matrice ainsi que la méthode d’application de compositions des faces sur un cube (nécessaire car affineTransform place toutes les images à plat.

-
Avec ces méthodes nous créons un cube dans GameView3D, qui est modifié en permanence en fonction des mouvements qui dépendent des tuiles. En plus de cela il faut

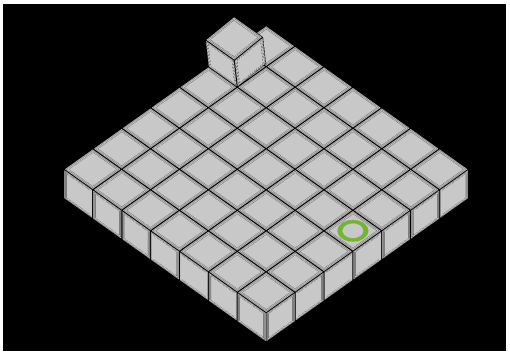
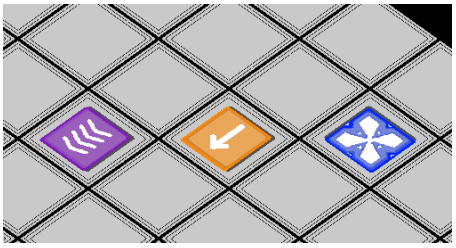
également calculer la norme des faces du cube pour voir avec la direction de la caméra quelles faces affichées.



<-cube sans vérification des faces caché


<-cube avec vérification des faces caché +images sur les faces

La grille est également composée de cubes mais comportant des images sur chaques faces (dessiné à la main). Et ces cubes ne nécessitent pas le calcul de face visible car inamovible.

Le gameView possède également les fonctions d'animation simple de perte et de victoire du jeu.